

“Sentimental Analysis for E-Commerce Products”

**A Minor Project Report Submitted in the Partial Fulfillment of
Requirement for the Award of Degree of
Bachelor of Technology
In
Computer science and Engineering**

Submitted To



**RAJIV GANDHI PROUDYOGIKI
VISHWAVIDYALAYA BHOPAL (M.P.)**

Submitted By :

**Lakhan patidar(0546CS191027)
Sanjay Kumar (0546ME191037)**

Submitted To:

**Mrs. Priya Pathak
(Asst. Prof. VITS BHOPAL)**

Under the supervision of:

**Prof. SUMIT SHARMA
(HOD CSE Department)**

Dept. – CSE, VITS Bhopal



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Vaishnavi Institute of Technology& Science Bhopal

Aug-Dec (2022)

VAISHNAVI INSTITUTE OF TECHNOLOGY & SCIENCE BHOPAL



CERTIFICATE

This is to certify that the dissertation entitled **“Sentimental Analysis for E-Commerce Products”** submitted to Rajiv Gandhi Proudhyogiki Vishwavidyalaya, Bhopal (M.P.) by

Lakhan Patidar (0546CS191027),
Sanjay Kumar (0546ME191037).

is a partial fulfillment of the requirement for the award of degree of **Bachelor of Technology** in computer science and Engineering. The matter embodied is the actual work done by (Lakhan Patidar (0546CS191027),

Sanjay Kumar (0546ME191037))

and is a record of bonafide work done by him under my supervision.

Dr. R. S. Sikarwar
(Director)
VITS, Bhopal

Prof. Sumit Sharma
(Supervisor & HOD, CSE)
VITS, Bhopal

ACKNOWLEDGEMENTS

Any assignment puts to litmus, test of individual's knowledge, credibility or experience and thus sole efforts of an individual are not sufficient to accomplish the desired task. Words shall never be able to describe neither the spirit with which we worked together nor shall they ever be able to express the feeling we felt towards our guides. Successful completion of a project involves interests and efforts of many people so it becomes obligatory on our part to extend our thanks to them.

I take this opportunity to thanks **Mrs.Priya Pathak (Asst.Prof. VITS Bhopal)** Supervisor and H.O.D. **Prof. Sumit Sharma VITS, Bhopal** for accepting me to work under his valuable guidance, closely supervising this work over the past few months and offering many innovative ideas and helpful suggestions as and when required. His valuable advice and support, was an inspiration and driving force for me. He has constantly enriched my raw ideas with his experience and knowledge. Indeed, it was a matter of great felicity to have worked under his aegis.

I would like to give thanks to **Dr. R. S. Sikarwar**, Director and Co-Supervisor **VITS, Bhopal** for his valuable guidance and motivation.

I also wish to thanks my respected teachers of **Computer Science, VITS**, for their constant support and guidance during my research work. I extend my gratitude to all teachers of department of Computer Science and colleagues, who have always been by my side, through thick and thin during these years and helped me in several ways.

Last but not least I'd like to thank "**The Almighty God**", my Parents, my family and my friends who directly or indirectly helped me in this endeavor.

NAME ENROLLMENT

Lakhan Patidar (0546CS191027)

Sanjay Kumar (0546ME191037)

CONTENTS

INTRODUCTION.....	1-2
SOFTWARE & HARDWARE REQUIREMENTS	3
SYSTEM DESIGN DFD, ER DIAGRAM.....	4-12
CODING	13-39
IMPLEMENTATION SNAPSHOT.....	40-49
TESTING	50
DEPLOYMENT	51-57
CONCLUSION.....	58
FUTURE SCOPE	59
GITHUB LINKS.....	60

INTRODUCTION

OBJECTIVE



The goal of the project is to carry out sentiment analysis on Amazon products dataset based on reviews on a million products and predict whether this review is 'Positive', 'Neutral', 'Negative'. Sentiment analysis is a text analysis method that detects polarity (e.g. a positive or negative opinion) within text, whether a whole document, paragraph, sentence, or clause. The reason for selecting this project is because understanding people's emotions is essential for businesses since customers can express their thoughts and feelings more openly than ever before. By automatically analyzing customer feedback, from survey responses to social media conversations, brands can listen attentively to their customers, and tailor products and services to meet their needs. For example, using sentiment analysis to automatically analyze 2,00,000+ reviews about your product could help you discover if customers are happy about your pricing plans and customer services.

- ❖ The advent of electronic commerce with growth in internet and network technologies has led customers to move to online retail platforms such as Amazon, Walmart, Target, etc. People often rely on customer reviews of products before they buy online. These reviews are often rich in information describing the product. Customers often choose to compare between various products and brands based on whether an item has a positive or negative review. More often, these reviews act as a feedback mechanism for the seller. Through this medium, sellers strategize their future sales and product improvement.
- ❖ Sentiment analysis or opinion mining is one of the major tasks of NLP (Natural Language Processing) that has gained much attention in recent years. The sentiment is a feeling, expression, thought, or judgment, and using sentiment analysis one can study the target audience's sentiments towards an entity. It's a form of text analysis that senses polarity (e.g. a positive or negative opinion) within whole text, sentence, paragraph or phrase. Knowing people's emotions is important for companies because consumers can communicate their thoughts and feelings more freely than ever before. With the technological improvements in the field of machine learning and automation, Companies can create system that automatically analyzing customer feedback, survey responses and social media interactions. These way marketers can listen to their customers closely and customize goods and services to suit their needs
- ❖ In this paper, we studied the notion of 'aboutness' through an important type of semantic processing task – 'topic modelling'. Most texts are usually comprised of multiple topics, i.e. topics being talked about in the text. For example, say a product manager at Amazon wants to understand what features of a recently released product (say Amazon Alexa) customers are talking about in their reviews. Say they are able to identify that 50% of people talk about the hardware, 30% talk about features related to music, while 20% talk about the packaging of the product. This would be immensely useful in a similar scenario where a seller on Amazon receives multiple negative reviews and wants to understand the reason behind it.

- ❖ We also studied the case where buyers are seldom misled by overall collective rating that Amazon without any bifurcation between a service review and product review. Hence, we trained our model to also classify the reviews as positive, negative and neutral.

Software & Hardware Requirements



Frontend and Backend : Streamlit,Heroku.



ML Recommender :ML Libraries like Pandas , NumPy ,seaborn, scikit Learn ,K Nearest Neighbor, Scipy, Bagging Boosting, Random Forest, Logistic Regression.



NLP Recommender : Bi-gram & Tri-gram Text-Pre-Processing, NER, NLTK, Word Cloud.



IDES USED : Jupyter Notebook, Pycharm, Visual Studio Code.

SYSTEM DESIGN DFD, ER DIAGRAM



Data Source: [boAt Rockerz 370 On Ear Bluetooth Headphones with Upto 12 Hours Playtime, Cozy Padded Earcups and Bluetooth v5.0, with Mic \(Buoyant Black\) : Amazon.in: Electronics](#)



- ❖ Our dataset comes from UCSD Design Lab [10]. It contains item reviews and metadata from Amazon, including 142.8 million reviews spreading over May 1996 - July 2014 in a one-review-per-line JSON format. This dataset incorporates reviews (ratings, content, support votes), item metadata (portrayals, class data, value, brand, and picture highlights), and connections. We have picked datasets for the following categories: Musical Instruments, Office Products, Home and Kitchen, Video Games, Beauty, Sports and Outdoors, Toys and Games, Cell Phones and Accessories, Health and Personal Care.
- ❖ The files contain attributes 'reviewer ID', 'ASIN', 'Reviewer Name', 'Reviewer text', 'Helpful', 'Summary', 'Rating', and 'Review time'. It demonstrates in that there are 5 group-rankings from 1 to 5. In comparison, these five classes are potentially imbalanced as Class 1 and Class 2 have limited quantities of data while Class 5 has over 500000 ratings.

- ❖ Data contains attributes such as reviewerID (which refers to the unique identification number for each reviewer who has bought the product from the website and provided a rating with some text review), reviewerName (gives the name of the reviewer), reviewText (this is the product review text written by the reviewer on the product page), summary (obtained from reviewText), overall (product rating on a scale of 1-5), asin (this is the ID of the product), helpful (helpfulness rating of the review, e.g. 2/3 5).

Data Preprocessing

- ❖ Data preprocessing is the process of transforming raw data into an understandable format.
- ❖ It is also an important step in data mining as we cannot work with raw data.
- ❖ The quality of the data should be checked before applying machine learning or data mining algorithms

Exploratory Data Analysis (EDA) :

Technically, The primary motive of EDA is to

- ☐ Examine the data distribution
- ☐ Handling missing values of the dataset(a most common issue with every dataset)
- ☐ Handling the outliers
- ☐ Removing duplicate data
- ☐ Encoding the categorical variables
- ☐ Normalizing and Scaling

Step 1 :

First, we will import all the python libraries that are required for this, which include NumPy for numerical calculations and scientific computing, Pandas for handling data, and Matplotlib and Seaborn for visualization.

Import numpy as np

Import pandas as pd

Import matplotlib as plt

Import seaborn as sns

Step 2 :

Then we will load the data into the **Pandas** data frame and beautifulsoup library for HTML parser. For this analysis,

```
soup = get_soup(f'https://www.amazon.in/Rockerz-370-Headphone-Bluetooth- Lightweight/product-reviews/B0856HNMR7/ref=cm_cr_ar_p_d_paging_btm_next_2?ie=UTF8&reviewerType=all_reviews&pageNumber')
```

Step 3 :

We can observe the dataset by checking a few of the rows using the **head()** method, which returns the first five records from the dataset.

```
Df.head()
```

Step 4 :

Using **shape**, we can observe the dimensions of the data.

```
Df.shape
```

Step 5 :

info() method shows some of the characteristics of the data such as Column Name, No. of non-null values of our columns, Dtype of the data, and Memory Usage.

```
amazon.info()
```

Step 6 :

We will use **describe()** method, which shows basic statistical characteristics of each numerical feature (int64 and float64 types): number of non-missing values, mean, standard deviation, range, median, 0.25, 0.50, 0.75 quartiles.

```
Amazon.describe()
```

Step 7 :

Handling missing values in the dataset. Luckily, this dataset doesn't have any missing values, but the real world is not so naive as our case.

So we removed a few values intentionally just to depict how to handle this particular case.

We can check if our data contains a null value or not by the following command

```
amazon.isnull().sum()
```

if we have found any missing values.

So, now we can handle the missing values by using a few techniques, which are

- ❑ **Drop the missing values** – If the dataset is huge and missing values are very few then we can directly drop the values because it will not have much impact.
- ❑ **Replace with mean values** – We can replace the missing values with mean values, but this is not advisable in case if the data has outliers.
- ❑ **Replace with median values** – We can replace the missing values with median values, and it is recommended in case if the data has outliers.
- ❑ **Replace with mode values** – We can do this in the case of a Categorical feature.
- ❑ **Regression** – It can be used to predict the null value using other details from the dataset.

Step 8 :

- ❖ The `nunique()` method returns the number of unique values for each column.
- ❖ By specifying the column axis (`axis='columns'`), the `nunique()` method searches column-wise and returns the number of unique values for each row.

```
df[.].unique
```

Step 9 :

- ❖ Python `iloc()` function enables us to select a particular cell of the dataset, that is, it helps us select a value that belongs to a particular row or column from a set of values of a data frame or dataset.
- ❖ With `iloc()` function, we can retrieve a particular value belonging to a row and column using the index values assigned to it.
- ❖ Remember, `iloc()` function accepts only integer type values as the index values for the values to be accessed and displayed.

```
Amazon = df.iloc[]
```

Step 10 :

- ❖ The `value_counts()` function is used to **get a Series containing counts of unique values**. The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default. `df['your_column'].value_counts()` - this will return the count of unique occurrences in the specified column.

```
Amazon[].value_counts()
```

LIBRARIES USED :

```
#importing packages
import requests
from bs4 import BeautifulSoup
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

Pandas :

Pandas is a Python library it is used to read the files.

Numpy :

Numpy is a python library used for working with arrays. It also has functions for working in domain of linear algebra transform, and matrices.

Mtplotlib :

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in python. Matplotlib makes easy things easy and hard things possible.

- ☐ Create Publication quality plots.
- ☐ Make interactive figures that can zoom, pan, update.
- ☐ Customize visual style and layout.
- ☐ Export to many file formats.
- ☐ Embed in JupyterLab and Graphical User Interfaces.
- ☐ Use a rich array of third-party-packages built on Matplotlib.

Seaborn :

seaborn is a data visualization library built on top of matplotlib and closely integrated with pandas data structures in python. visualization is the central part of seaborn helps in exploration and understanding of data

Seaborn offers the following functionalities.

1. Dataset oriented APL to determine the relationship between variables.
2. Automatic estimation and plotting of linear regression plots.
3. It support high-level abstractions for multi-plot grids.
4. Visualizing univariate and bivariate distribution.

SciPy :

SciPy is a scientific computation library that uses NumPy underneath. SciPy stands for Scientific Python. It provides more utility functions for optimization, stats and signal processing. Like NumPy, SciPy is open source so we can use it freely.

Sklearn:

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in python. It provides a selection of efficient tools for machine learning and statical modelling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python.

Beautiful Soup:

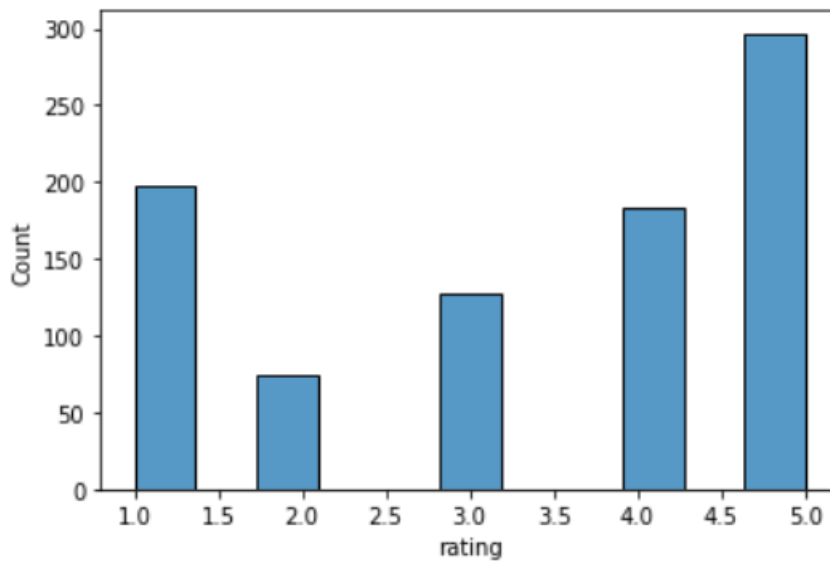
In today's world, we have tons of unstructured data/information (mostly web data) available freely. Sometimes the freely available data is easy to read and sometimes not. No matter how your data is available, web scraping is very useful tool to transform unstructured data into structured data that is easier to read & analyze. In other words, one way to collect, organize and analyze this enormous amount of data is through web scraping. So let us first understand what is web-scraping.

Visualization :

Histogram:

The Histogram is a representation of the numerical data, not accurate but an estimate. Karl Pearson was the first one to introduce the idea of Histogram. To create the Histogram the first step is binning, which is also called data binning, or bucketing or discrete binning. In this step the data is pre-processed and used for reducing the effects of minor observation errors, it divides the whole [range](#) of values into a sequence of intervals, and then counts the number of values that falls into each of the intervals.

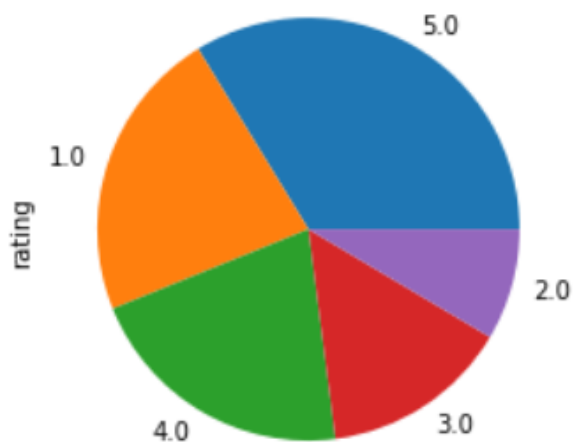
```
sns.histplot(amazon['rating'])  
plt.show()
```



Piechart :

A pie chart is a type of graph that represents the data in the circular graph. The slices of pie show the relative size of the data, and it is a type of pictorial representation of data. A pie chart requires a list of categorical variables and numerical variables.

```
amazon.rating.value_counts().plot(kind='pie')  
plt.show()
```



Text-pre-processing :

As we know Machine Learning needs data in the numeric form. We basically used encoding technique (BagOfWord, Bi-gram,n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and *this process to prepare(or clean) text data before encoding is called text preprocessing*, this is the very first step to solve the NLP problems.

Removing Stopwords :

- ❖ On removing stopwords, dataset size decreases and the time to train the model also decreases.
- ❖ Removing stopwords can potentially help improve the performance as there are fewer and only meaningful tokens left. Thus, it could increase classification accuracy.
- ❖ Even search engines like Google remove stopwords for fast and relevant retrieval of data from the database.

CODING

□ All Libraries used.

```
: #importing packages
import requests
from bs4 import BeautifulSoup
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

□ HTML PARSER.

```
#using requests which allows us to send HTTP requests using Python
def get_soup(url):
    url = url
    page = requests.get(url)
    soup = BeautifulSoup(page.content, 'html.parser')
    return soup
```

```
reviewlist = []
#fetching only data which is necessary like rating title and content given by the customers
def get_reviews(soup):
    reviews = soup.find_all('div', {'data-hook': 'review'})
    try:
        for item in reviews:
            review = {
                'title': item.find('a', {'data-hook': 'review-title'}).text.strip(),
                'rating': float(item.find('i', {'data-hook': 'review-star-rating'}).text.replace('out of 5 stars', '').strip()),
                'content': item.find('span', {'data-hook': 'review-body'}).text.strip(),
            }
            reviewlist.append(review)
    except:
        pass
```

□ **DATA LINK** - [boAt Rockerz 370 On Ear Bluetooth Headphones with Upto 12 Hours Playtime, Cozy Padded Earcups and Bluetooth v5.0, with Mic \(Buoyant Black\) : Amazon.in: Electronics](https://www.amazon.in/Rockerz-370-Headphone-Bluetooth-Lightweight/product-reviews/B0856HNMR7/ref=cm_cr_arp_d_paging_btm_next_2?ie=UTF8&reviewerType=all_reviews&pageNumber={x})

➤ **Import data using BeautifulSoup library.(BS4).**

#creating a loop from 1 to 100 reviews pages of the product

for x in range(1,100):

```
soup = get_soup(f'https://www.amazon.in/Rockerz-370-Headphone-Bluetooth-Lightweight/product-reviews/B0856HNMR7/ref=cm_cr_arp_d_paging_btm_next_2?ie=UTF8&reviewerType=all_reviews&pageNumber={x}')
```

```
print(f'Getting page: {x}')
```

```
get_reviews(soup)
```

```
print(len(reviewlist))
```

```
if not soup.find('li', {'class': 'a-disabled a-last'}):
```

```
    pass
```

```
else:
```

```
    break
```

```
print('Done')
```

```
Getting page: 1
10
Getting page: 2
20
Getting page: 3
30
Getting page: 4
40
Getting page: 5
50
Getting page: 6
50
Getting page: 7
60
Getting page: 8
70
Getting page: 9
70
Getting page: 10
70
```

➤ Creating Data Frame

```
#creating dataframe of the list
df = pd.DataFrame(reviewlist)
df
```

	title	rating	content
0	I am very happy to buy this product...	5.0	Nice product... 🍌 🍌 ..
1	Amazingly Amazon	4.0	It says 8 hrs but since I use this to watch le...
2	base king	4.0	the base is really good. it a value for this r...
3	The head phone plastic band broken out.	1.0	This product was had manufacturing defect insi...
4	Cheap Plastic, Average product. Sound is dist...	2.0	Edit: After 1 year of usage.The headphone star...
...	...	...	...
875	Good	4.0	The media could not be loaded.\n ...
876	Wonderfull	5.0	Very good quality and bass
877	Excellent	5.0	Excellent product
878	Fast delivery and jenuie Products from Amezon ❤️ ❤️	5.0	Nice Sound Quality With a High Frequency of ba...
879	Good headphone at this price ..build material ...	3.0	Its good...nice product...but my headphone aft...

880 rows x 3 columns

➤ Data Pre-Processing.

Data preprocessing is the process of transforming raw data into an understandable format.

It is also an important step in data mining as we cannot work with raw data.

The quality of the data should be checked before applying machine learning or data mining algorithms

```
: amazon.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 880 entries, 0 to 879
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype  
---  ---
0    rating      880 non-null    float64
1    reviews     880 non-null    object  
dtypes: float64(1), object(1)
memory usage: 13.9+ KB
```

```
: #checking null values
amazon.isnull().sum()
```

```
: rating      0
reviews      0
dtype: int64
```

- Data Exploration and Cleaning - The first step in any analysis should start with data understanding and this can be done with EDA process.

➤ Before we dive into data visualizations, let's get a look on the descriptive statistics from the Data set.

```
df.shape
```

```
(880, 3)
```

```
df['content'].unique
```

```
<bound method Series.unique of 0
1    It says 8 hrs but since I use this to watch le...
2    the base is really good. it a value for this r...
3    This product was had manufacturing defect insi...
4    Edit: After 1 year of usage.The headphone star...
...
875    The media could not be loaded.\n
876    Very good quality and bass
877    Excellent product
878    Nice Sound Quality With a High Frequency of ba...
879    Its good...nice product...but my headphone aft...
Name: content, Length: 880, dtype: object>
```

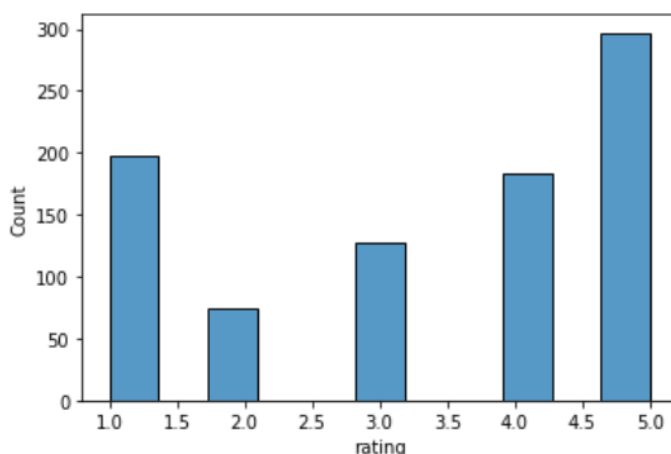
Nice product...👍👍..

➤ Data Visualization for Ratings.

Histogram:

The Histogram is a representation of the numerical data, not accurate but an estimate. Karl Pearson was the first one to introduce the idea of Histogram. To create the Histogram the first step is binning, which is also called data binning, or bucketing or discrete binning. In this step the data is pre-processed and used for reducing the effects of minor observation errors, it divides the whole [range](#) of values into a sequence of intervals, and then counts the number of values that falls into each of the intervals.

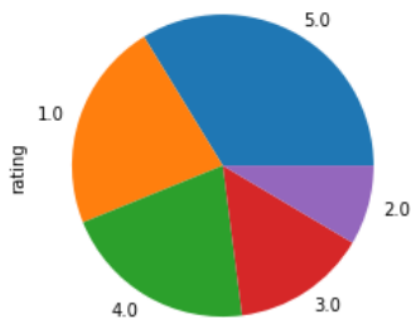
```
sns.histplot(amazon['rating'])
plt.show()
```



Piechart :

A pie chart is a type of graph that represents the data in the circular graph. The slices of pie show the relative size of the data, and it is a type of pictorial representation of data. A pie chart requires a list of categorical variables and numerical variables.

```
amazon.rating.value_counts().plot(kind='pie')
plt.show()
```



Text Preprocessing

#to analyze the data we are replicating the dataframe as amazon to store more columns but it won't disturb the main dataset
`amazon_analysis = amazon.copy()`

#Number of characters in single tweet
`amazon_analysis['char_count'] = amazon_analysis['reviews'].str.len() ## this also includes spaces`
`amazon_analysis[['reviews', 'char_count']]`

	reviews	char_count
0	I am very happy to buy this product... Nice p...	59
1	Amazingly Amazon It says 8 hrs but since I us...	239
2	base king the base is really good. it a value...	126
3	The head phone plastic band broken out. This ...	527
4	Cheap Plastic, Average product. Sound is dist...	2218
...	...	...
875	Good The media could not be loaded.In ...	146
876	Wonderfull Very good quality and bass	38
877	Excellent Excellent product	28
878	Fast delivery and jenu Products from Amezon♥...	212
879	Good headphone at this price ..build material ...	196

880 rows × 2 columns

#count of special charaters in the text

```
amazon_analysis['punctuations'] = amazon_analysis['reviews'].apply(lambda x: len([x for x in x.split() if
x.startswith('!'#$%&'()*+,-./:;<=>?@[^_`{|}~'])))
amazon_analysis[['reviews','punctuations']]
```

	reviews	punctuations
0	I am very happy to buy this product... Nice p...	0
1	Amazingly Amazon It says 8 hrs but since I us...	0
2	base king the base is really good. it a value...	0
3	The head phone plastic band broken out. This ...	0
4	Cheap Plastic, Average product. Sound is dist...	0
...	...	...
875	Good The media could not be loaded.\n ...	0
876	Wonderful! Very good quality and bass	0
877	Excellent Excellent product	0
878	Fast delivery and jenu Products from Amezon♥...	0
879	Good headphone at this price ..build material ...	0

880 rows × 2 columns

```
#number of numericals present in each row
amazon_analysis['numerics'] = amazon_analysis['reviews'].apply(lambda x: len([x for x in x.split() if x.isdigit()]))
amazon_analysis[['reviews','numerics']]
```

	reviews	numerics
0	I am very happy to buy this product... Nice p...	0
1	Amazingly Amazon It says 8 hrs but since I us...	2
2	base king the base is really good. it a value...	0
3	The head phone plastic band broken out. This ...	1
4	Cheap Plastic, Average product. Sound is dist...	4
...	...	...
875	Good The media could not be loaded.\n ...	0
876	Wonderfull! Very good quality and bass	0
877	Excellent Excellent product	0
878	Fast delivery and jenu Products from Amezon♥...	0
879	Good headphone at this price ..build material ...	1

880 rows × 2 columns

- **NLTK** - NLTK (Natural Language Toolkit) is the go-to API for NLP (Natural Language Processing) with Python. It is a really powerful tool to preprocess text data for further analysis like with ML models for instance. It helps convert text into numbers, which the model can then easily work with.

```
import nltk
nltk.download('stopwords')

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\91960\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

True

```
from nltk.corpus import stopwords
```

```
#number of stopwords in each tweet
stop = stopwords.words('english')
```

```
amazon_analysis['stopwords'] = amazon_analysis['reviews'].apply(lambda x: len([x for x in x.split() if x in stop]))
amazon_analysis[['reviews', 'stopwords']]
```

	reviews	stopwords
0	I am very happy to buy this product... Nice p...	4
1	Amazingly Amazon It says 8 hrs but since I us...	13
2	base king the base is really good. it a value...	12
3	The head phone plastic band broken out. This ...	33
4	Cheap Plastic, Average product. Sound is dist...	128
...	...	...
875	Good The media could not be loaded.\n ...	3
876	Wonderful! Very good quality and bass	1
877	Excellent Excellent product	0
878	Fast delivery and jenuue Products from Amezon♥...	9
879	Good headphone at this price ..build material ...	12

880 rows × 2 columns

Removing Stopwords :

- ❖ On removing stopwords, dataset size decreases and the time to train the model also decreases.
- ❖ Removing stopwords can potentially help improve the performance as there are fewer and only meaningful tokens left. Thus, it could increase classification accuracy.
- ❖ Even search engines like Google remove stopwords for fast and relevant retrieval of data from the database.

```
: #removing all the stopwords in the column
stop = stopwords.words('english')
amazon['reviews'] = amazon['reviews'].apply(lambda x: " ".join(x for x in x.split() if x not in stop))
amazon['reviews']

<ipython-input-24-c4d734e2072a>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
amazon['reviews'] = amazon['reviews'].apply(lambda x: " ".join(x for x in x.split() if x not in stop))

0          I happy buy product... Nice product... 🍌 🍌 ..
1    Amazingly Amazon It says 8 hrs since I use wat...
2    base king base really good. value range money....
3    The head phone plastic band broken out. This p...
4    Cheap Plastic, Average product. Sound distorte...
...
875    Good The media could loaded. Kam Achcha kar ra...
876           Wonderful! Very good quality bass
877           Excellent Excellent product
878    Fast delivery jenu Products Amezom ❤️ ❤️ Nice S...
879    Good headphone price ..build material good eno...
Name: reviews, Length: 880, dtype: object
```

```
: amazon['reviews'].isnull().sum()
```

```
: 0
```

```
: #converting all the upper case and sentence case in lower case
amazon = amazon.apply(lambda x: x.astype(str).str.lower())
```

```
: #converted in lower case
amazon.head()
```

	rating	reviews
0	5.0	i happy buy product... nice product... 🍌 🍌 ..
1	4.0	amazingly amazon it says 8 hrs since i use wat...
2	4.0	base king base really good. value range money....
3	1.0	the head phone plastic band broken out. this p...
4	2.0	cheap plastic, average product. sound distorte...

➤ RARE WORD COUNT & REMOVE

```
#rare words counts
freq = pd.Series(' '.join(amazon['reviews']).split()).value_counts()[-10:]
freq
```

```
anything.      1
reconnect      1
class....,     1
binging        1
24             1
best 100       1
gaming.i       1
isme           1
sec            1
quality.4.     1
dtype: int64
```

```
#rare words removal
freq = list(freq.index)
amazon['reviews'] = amazon['reviews'].apply(lambda x: " ".join(x for x in x.split() if x not in freq))
amazon['reviews'].head()
```

```
0      i happy buy product... nice product... 🍌 🍌 ..
1  amazingly amazon it says 8 hrs since i use wat...
2  base king base really good. value range money....
3  the head phone plastic band broken out. this p...
4  cheap plastic, average product. sound distorte...
Name: reviews, dtype: object
```

```
amazon
```

	rating	reviews
0	5.0	i happy buy product... nice product... 🍌 🍌 ..
1	4.0	amazingly amazon it says 8 hrs since i use wat...
2	4.0	base king base really good. value range money....
3	1.0	the head phone plastic band broken out. this p...
4	2.0	cheap plastic, average product. sound distorte...
...	...	...
875	4.0	good the media could loaded. kam achcha kar ra...
876	5.0	wonderfull very good quality bass
877	5.0	excellent excellent product
878	5.0	fast delivery jenue products amezon ❤️ ❤️ nice s...
879	3.0	good headphone price ..build material good eno...

➤ REMOVE ALL TYPE OF EMOJIS

```
import re
```

```
#removing all the emojis present in the text
```

```
def remove_emoji(text):
```

```
    emoji_pattern = re.compile("[
```

```
        u"\U0001F600-\U0001F64F"    # emoticons
```

```
        u"\U0001F300-\U0001F5FF"    # symbols & pictographs
```

```
        u"\U0001F680-\U0001F6FF"    # transport & map symbols
```

```
        u"\U0001F1E0-\U0001F1FF"    # flags
```

```
        u"\U00002702-\U000027B0"
```

```
        u"\U000024C2-\U0001F251"
```

```
    "]+", flags=re.UNICODE)
```

```
    return emoji_pattern.sub(r'', text)
```

```
amazon['reviews'] = amazon['reviews'].apply(lambda x: remove_emoji(x))
```

```
amazon['reviews']
```

```
0          i happy buy product... nice product.....
```

```
1    amazingly amazon it says 8 hrs since i use wat...
```

```
2    base king base really good. value range money....
```

```
3    the head phone plastic band broken out. this p...
```

```
4    cheap plastic, average product. sound distorte...
```

```
...
```

```
875    good the media could loaded. kam achcha kar ra...
```

```
876          wonderful! very good quality bass
```

```
877          excellent excellent product
```

```
878    fast delivery jenu products amezon nice sound...
```

```
879    good headphone price ..build material good eno...
```

```
Name: reviews, Length: 880, dtype: object
```

STEMMING - In NLP, stemming is a technique for normalizing words. It is a method of converting a group of sentence words into a sequence in order to reduce the time it takes to look up the information. The words that have the same meaning but differ due to context or sentence are normalized Using this method.

```
: from nltk.stem import PorterStemmer
st = PorterStemmer()
amazon['reviews'][:5].apply(lambda x: " ".join([st.stem(word) for word in x.split()]))

0      i happi buy product... nice product.....
1  amazngli amazon it say 8 hr sinc i use watch ...
2  base king base realli good. valu rang money.i ...
3  the head phone plastic band broken out. thi pr...
4  cheap plastic, averag product. sound distort h...
Name: reviews, dtype: object
```

```
: from textblob import Word
from textblob import TextBlob
```

- **LEMMATIZATION** - Lemmatization is a text normalization technique used in Natural Language Processing (NLP), that switches any kind of a word to its base root mode. Lemmatization is responsible for grouping different inflected forms of words into the root form, having the same meaning.

```
amazon['reviews'] = amazon['reviews'].apply(lambda x: " ".join([Word(word).lemmatize() for word in x.split()]))
amazon['reviews'].head()

0      i happy buy product... nice product.....
1  amazingly amazon it say 8 hr since i use watch...
2  base king base really good. value range money...
3  the head phone plastic band broken out. this p...
4  cheap plastic, average product. sound distorte...
Name: reviews, dtype: object
```

- **N_Gram** - Basically, an N-gram model predicts the occurrence of a word based on the occurrence of its $N - 1$ previous words. So here we are answering the question – how far back in the history of a sequence of words should we go to predict the next word? For instance, a bigram model ($N = 2$) predicts the occurrence of a word given only its previous word (as $N - 1 = 1$ in this case).

```
#bigrams
TextBlob(amazon['reviews'] [0]).ngrams(2)
```

```
[WordList(['i', 'happy']),
 WordList(['happy', 'buy']),
 WordList(['buy', 'product']),
 WordList(['product', 'nice']),
 WordList(['nice', 'product'])]
```

```
#trigrams
TextBlob(amazon['reviews'] [0]).ngrams(3)
```

```
[WordList(['i', 'happy', 'buy']),
 WordList(['happy', 'buy', 'product']),
 WordList(['buy', 'product', 'nice']),
 WordList(['product', 'nice', 'product'])]
```

CountVectorizer - CountVectorizer tokenizes (tokenization means breaking down a sentence or paragraph or any text into words) the text along with performing very basic preprocessing like removing the punctuation marks, converting all the words to lowercase, etc. The vocabulary of known words is formed which is also used for encoding unseen text later.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
cv=CountVectorizer()
reviewcv=cv.fit_transform(amazon['reviews'])
print(cv.get_feature_names())
```

```
['0v', '10', '100', '1000', '1000000', '1000first', '1000rsthere', '1000then', '100rs', '1079rupees', '1099', '10bass', '10bl
uetooth', '10build', '10durability', '10hrs', '10m', '10mic', '10overall', '10sec', '10sound', '10total', '1100', '1100rs',
'1129', '1199', '12', '1200', '1200rs', '1200rupees', '1299', '13992', '14', '14th', '15', '1500', '15days', '15hrs', '15th',
'18', '18th', '19', '19th', '1hr', '1k', '1month', '1or', '1st', '1year', '20', '200', '2020', '2021', '2021didn', '21', '2
2', '22nd', '23', '24th', '25', '26th', '27', '27th', '29', '2hr', '2hrs', '2hrs2', '2k', '2nd', '30', '300', '3000', '30hr',
'30rs', '310', '320', '350', '370', '370r', '38', '3daysi', '3hrs', '3rd', '40', '400', '40mm', '450', '450note', '45min', '4
6', '4nov', '4th', '50', '500', '50if', '510', '5bass', '5bass4', '5battery', '5comfort', '5connectivity', '5ears', '5k', '5m
aterial', '5mic', '5mm', '5noise', '5overall', '5recommended', '5sound', '5thank', '5to6', '60', '600', '60mts', '67', '6day
s', '6months', '6th', '70', '799in', '7days', '7m', '7mic', '80', '800for', '849', '899', '899rs', '8d', '8hrs', '8t', '8%',
'90', '900', '98101', '98127', '999', 'a50', 'aache', 'aalo', 'aaye', 'ab', 'abd', 'abhi', 'able', 'abour', 'above', 'absolut
e', 'absolutely', 'abt', 'abv', 'ac', 'accept', 'acceptable', 'accepted', 'accessories', 'accha', 'accidentally', 'accordin
g', 'accurate', 'acha', 'achcha', 'ache', 'achi', 'aching', 'acording', 'acoustics', 'acpected', 'across', 'activate', 'actu
ally', 'ad', 'add', 'added', 'adjust', 'adjustable', 'admirable', 'adults', 'advantage', 'advantages', 'advertising', 'advic
e', 'advise', 'advised', 'affordable', 'afordable', 'after', 'aftermarket', 'aftersales', 'ag', 'again', 'age', 'agent', 'ag
o', 'ahead', 'ain', 'aint', 'air', 'airspace', 'alert', 'alibaba', 'all', 'allowed', 'allowing', 'almost', 'along', 'alos',
'alot', 'already', 'alright', 'also', 'alsobut', 'alsogood', 'alternate', 'alternative', 'although', 'altogether', 'always',
'am', 'aman', 'amateur', 'amazing', 'amazingjust', 'amazingly', 'amazon', 'amezon', 'amount', 'amplitude', 'an', 'anc', 'an
d', 'android', 'angle', 'annoying', 'another', 'answering', 'any', 'anybody', 'anyhow', 'anyone', 'anyother', 'anything', 'an
ytime', 'anyways', 'apart', 'apex', 'apne', 'app', 'apparently', 'appeared', 'appearing', 'appendage', 'appended', 'apperanc
```

```

cv = CountVectorizer()

reviewcv = cv.fit_transform(amazon['reviews'] )
sum_words = reviewcv.sum(axis=0)
words_freq = [(word, sum_words[0, idx]) for word, idx in cv.vocabulary_.items()]
words_freq = sorted(words_freq, key = lambda x: x[1], reverse=True)
wf_df = pd.DataFrame(words_freq)
wf_df.columns = ['words', 'count']

pd.options.display.max_rows=None
wf_df

```

	words	count
0	good	729
1	product	660
2	quality	620
3	sound	487
4	it	359
5	headphone	306
6	the	260
7	bass	197
8	ear	181
9	buy	169
10	use	169
11	one	169

- **CountVectorizer with Bi-gram & Tri-gram** - Bigram Trigram and NGram in NLP, How to calculate the unigram, bigram, trigram, and ngram probabilities of a sentence? Maximum likelihood estimation to calculate the ngram probabilities

Bi-gram

```

#Bi-gram
def get_top_n2_words(corpus, n=None):
    vec1 = CountVectorizer(ngram_range=(2,2),
                           max_features=2000).fit(corpus)
    bag_of_words = vec1.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in
                   vec1.vocabulary_.items()]
    words_freq = sorted(words_freq, key = lambda x: x[1],
                        reverse=True)
    return words_freq[:n]

```

```

top2_words = get_top_n2_words(amazon['reviews'], n=5000)
top2_df = pd.DataFrame(top2_words)
top2_df.columns=["Bi-gram", "Freq"]
top2_df

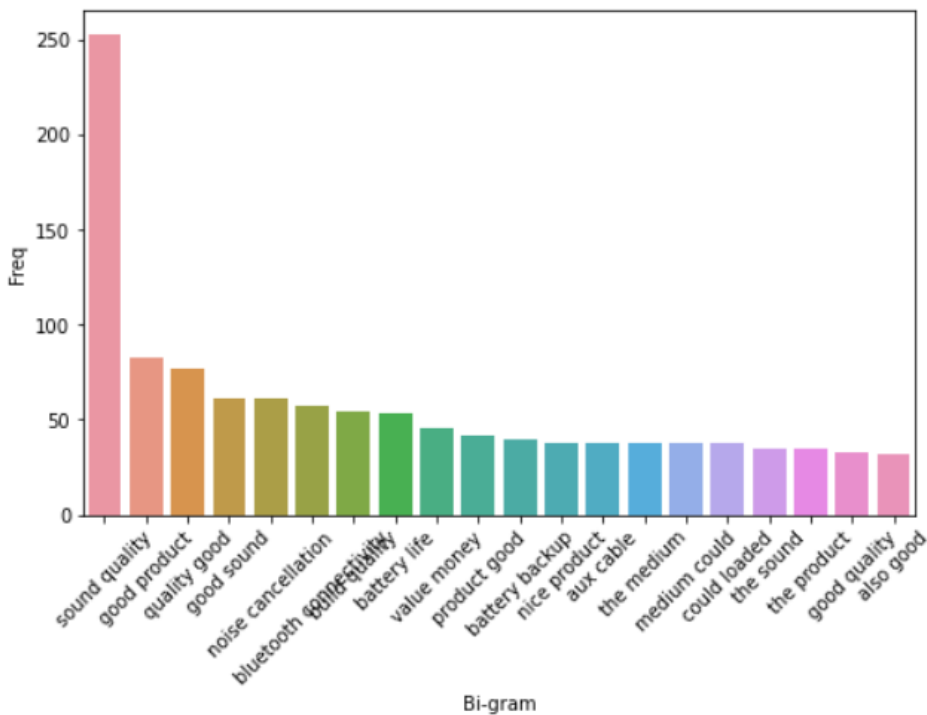
```

	Bi-gram	Freq
0	sound quality	253
1	good product	83
2	quality good	77
3	good sound	61
4	noise cancellation	61
5	bluetooth connectivity	57
6	build quality	54
7	battery life	53
8	value money	45
9	product good	42
10	battery backup	40
11	nice product	38

```

: import matplotlib.pyplot as plt
import seaborn as sns
top20_bigram = top2_df.iloc[0:20,:]
fig = plt.figure(figsize = (8, 5)) #figure size of the visualization
plot=sns.barplot(x=top20_bigram["Bi-gram"],y=top20_bigram["Freq"])
plot.set_xticklabels(rotation=45,labels = top20_bigram["Bi-gram"])
plt.show()

```



Tri-gram

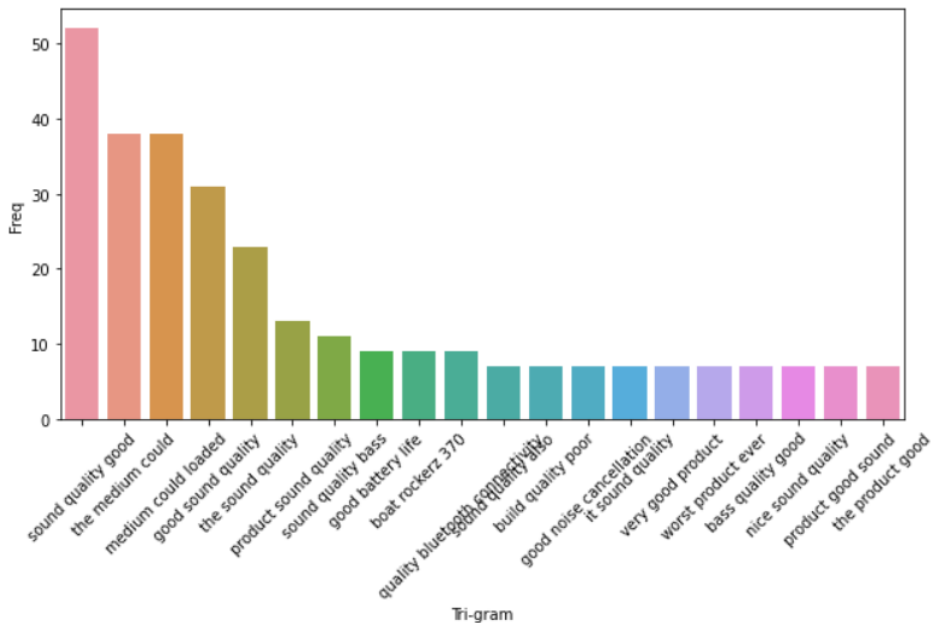
```
def get_top_n3_words(corpus, n=None):
    vec1 = CountVectorizer(ngram_range=(3,3),
                           max_features=2000).fit(corpus)
    bag_of_words = vec1.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in
                   vec1.vocabulary_.items()]
    words_freq = sorted(words_freq, key = lambda x: x[1],
                        reverse=True)
    return words_freq[:n]
```

```
top3_words = get_top_n3_words(amazon['reviews'] , n=5000)
top3_df = pd.DataFrame(top3_words)
top3_df.columns=["Tri-gram", "Freq"]

top3_df
```

	Tri-gram	Freq
0	sound quality good	52
1	the medium could	38
2	medium could loaded	38
3	good sound quality	31
4	the sound quality	23
5	product sound quality	13
6	sound quality bass	11
7	good battery life	9
8	boat rockerz 370	9
9	quality bluetooth connectivity	9
10	sound quality also	7
11	build quality poor	7

```
import seaborn as sns
top20_trigram = top3_df.iloc[0:20,:]
fig = plt.figure(figsize = (10, 5))
plot=sns.barplot(x=top20_trigram["Tri-gram"],y=top20_trigram["Freq"])
plot.set_xticklabels(rotation=45,labels = top20_trigram["Tri-gram"])
plt.show()
```



➤ Named Entity Recognition (NER) :

- ❖ The Named Entity Recognition in NLP, or NER, is to detect and label these nouns with the real-world concepts that they represent. But NER systems aren't just doing a simple dictionary lookup. Instead, they are using the context of how a word appears in the sentence and a statistical model to guess which type of noun a word represents.
- ❖ In natural language processing, named entity recognition (NER) is the problem of recognizing and extracting specific types of entities in text. Such as *people* or *place* names. In fact, any concrete "thing" that has a name. At any level of specificity. *Job titles, public school names, sport names, music...*

```
import string
import re #regular expression
import spacy
```

```
nlp = spacy.load("en_core_web_sm")

one_block = str(amazon['reviews'])
doc_block = nlp(one_block)
spacy.displacy.render(doc_block, style='ent', jupyter=True)
```

0 **CARDINAL** i happy buy product... nice product....

1 amazingly amazon it say 8 **CARDINAL** hr since i use watch...

2 base king base really good. value range money....

3 the head phone plastic band broken out. this p...

4 cheap plastic, average product. sound distorte...

5 value money sound crystal clearbass pretty goo...

6 **PERCENT** don't buy please... bass **PERSON** disgusting... range g...

7 cheap quality i used 1 month **DATE** headphone literal...

8 **PERCENT** after 2 month **DATE** review marvelous performanceif w...

#nouns and verbs in the text

```
nouns_verbs=[token.text for token in doc_block if token.pos_ in ('NOUN','VERB')]
print(nouns_verbs)
```

```
['buy', 'product', 'product', 'amazon', 'say', 'hr', 'use', 'watch', 'base', 'king', 'base', 'value', 'head', 'phone', 'plast
ic', 'band', 'broken', 'p', 'plastic', 'product', 'distorte', 'value', 'money', 'sound', 'crystal', 'clearbass', 'buy', 'bas
s', 'quality', 'used', 'month', 'headphone', 'month', 'review', 'performanceif', 'broken', 'month', 'fan', 'boat', 'product',
'recieved', 'used', 'product', 'br', 'go', 'boat', 'product', 'month', 'value', 'money', 'product', 'products', 'buy', 'produ
ct', 'using', 'h', 'worry', 'writing', 'review', 'mo', 'boat', 'rocker', 'headphone', 'medium', 'loaded', 'headphones', 'head
phones', 'broke', 'month', 'return', 'warranty', 'period', 'pro', 'price', 'product', 'voice', 'cancellation', 'ear', 'head',
'stared', 'paining', 'con', 'like', 'start', 'sound', 'batt', 'reading', 'review', 'people', 'l', 'mind', 'blowing', 'produc
t', 'wish', 'give', 'star', 'pr', 'problem', 'headphone', 'term', 'quality', 'build', 'quality', 'ergonomics', 'bought', 'mon
th', 'ago.work', 'bluetooth', 'connectivity', 'sound', 'quality', 'doubt', 'goo', 'product', 'purchase', 'price(999', 'bough
t', 'attend', 'class', 'c', 'product', 'battery', 'life', 'using', 'month', 'product', 'price', 'blu', 'pain', 'ear', 'usin
g', 'headphone', 'work', 'sound', 'quality', 'crystal', 'cl', 'waste', 'money', 'believe', 'noise', 'cancellation', 'son', 'h
ear', 'working', '1st', 'week', 'started', 'issue', 'month', 'using', 'p', 'headphone', 'quality', 'product', 'price', 'versi
on', 'price', 'product', 'come', 'sound', 'longtime', 'ache', 'ear', 'cause', 'product', 'price', 'range', 'medium', 'load',
'plastic', 'product', 'broken', 'working', 'bought', 'bass', 'quality', 'headphone', 'sound', 'moment', 'product', 'listenin
g', 'music', 'walking', 'trouble', 'buy', 'budget', 'product1', 'good', 'thing', 'opening', 'pac', 'headphone', 'price', 'ran
ge', 'headphone', 'review', 'read', 'buying', 'headset', 'goin', 'quality', 'choice', 'price', 'writing', 'review', 'usin',
'battery', 'champ', 'bought', 'thing', 'quality', 'crystal', 'definit', 'build', 'quality', 'product', 'build', 'quality',
'b', 'product', 'waste', 'money', 'earcups', 'cushion', 'detective', 'piece', 'received', 'detective', 'piece', 'recei', 'mus
ic', 'lover', 'quality', 'music', 'boat', 'rockerz', 'headset', 'product', 'product', 'switch', 'put', 'headphone', 'qualit
```

```
len(nouns_verbs)
```

3134

```
#Counting the noun & verb tokens
from sklearn.feature_extraction.text import CountVectorizer
cv=CountVectorizer()
#with collected nouns and verbs
X=cv.fit_transform(nouns_verbs)
sum_words=X.sum(axis=0)

words_freq=[(word,sum_words[0,idx]) for word,idx in cv.vocabulary_.items()]
words_freq=sorted(words_freq, key=lambda x: x[1], reverse=True)

wd_df=pd.DataFrame(words_freq)
wd_df.columns=['words','count']
wd_df
```

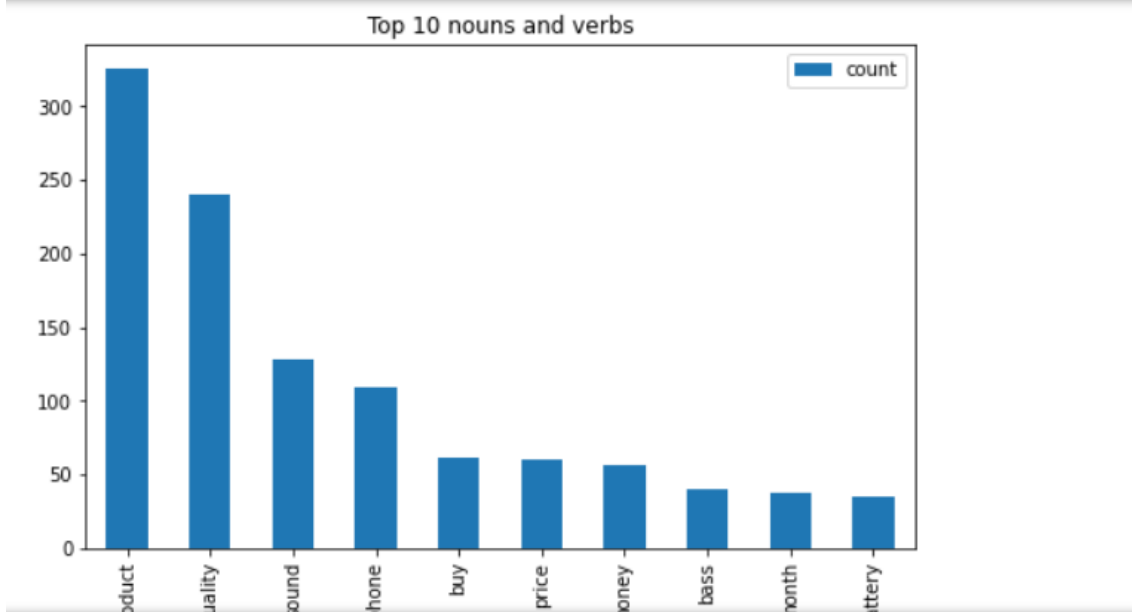
24	range	19
25	used	18
26	connectivity	18
27	noise	17
28	plastic	16
29	review	16
30	waste	16
31	music	16
32	life	15
33	headset	15
34	like	14
35	thing	14
36	head	13

➤ BARCHART :

- ❖ A bar chart is a pictorial representation of data that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. In other words, it is the pictorial representation of dataset. These data sets contain the numerical values of variables that represent the length or height.
- ❖ **R** uses the function **barplot()** to create bar charts. Here, both vertical and Horizontal bars can be drawn.

```
# Visualizing results (Barchart for top 10[nouns + verbs])
```

```
wd_df[0:10].plot.bar(x='words',figsize=(8,5),title='Top 10 nouns and verbs');  
plt.show()
```



➤ WORD CLOUD :

Is word cloud a part of NLP? Natural language processing (NLP) lies at the intersection between linguistics and computer science. A word cloud is a snapshot of a text.

Word Cloud

```
from PIL import Image
```

Define a function to plot word cloud

```
def plot_cloud(wordcloud): plt.figure(figsize=(20,30)) plt.imshow(wordcloud) plt.axis('off')
```

Generate Word Cloud

```
from wordcloud import WordCloud, STOPWORDS  
  
STOPWORDS.add('words') STOPWORDS.add('rt') STOPWORDS.add('yeah') mask = np.array(Image.open("D:\DATA SCIENCE\Project\Sentiment Analysis  
on Amazon Product Reviews\amazon-icon-6.png")) wordcloud = WordCloud(width=10000,height=5000,background_color='white',max_words=500,  
colormap='Set1', mask=mask, stopwords=STOPWORDS).generate(str(wd_df)) plt.savefig("amazon.png", format="png") plot_cloud(wordcloud) plt.show()
```

Sentiment Analysis for each word

```
]: #sentiment
wd_df['words'][:5].apply(lambda x: TextBlob(x).sentiment)
wd_df['sentiment'] = wd_df['words'].apply(lambda x: TextBlob(x).sentiment[0] )
wd_df[['words', 'sentiment']]
```

```
]:
```

	words	sentiment
0	product	0.000000
1	quality	0.000000
2	sound	0.400000
3	headphone	0.000000
4	buy	0.000000
5	price	0.000000
6	money	0.000000
7	bass	-0.150000
8	month	0.000000
9	battery	0.000000
10	value	0.000000
11	ear	0.000000

```
# subjectivity and polarity
from textblob import TextBlob
def getSubjectivity(clean_review):
    return TextBlob(clean_review).sentiment.subjectivity

def getPolarity(clean_review):
    return TextBlob(clean_review).sentiment.polarity

wd_df['Subjectivity'] = wd_df['words'].apply(getSubjectivity)
wd_df['Polarity'] = wd_df['words'].apply(getPolarity)
```

```
wd_df
```

	words	count	sentiment	Subjectivity	Polarity
0	product	326	0.000000	0.000000	0.000000
1	quality	240	0.000000	0.000000	0.000000
2	sound	128	0.400000	0.400000	0.400000
3	headphone	109	0.000000	0.000000	0.000000
4	buy	61	0.000000	0.000000	0.000000
5	price	60	0.000000	0.000000	0.000000
6	money	56	0.000000	0.000000	0.000000
7	bass	40	-0.150000	0.500000	-0.150000
8	month	38	0.000000	0.000000	0.000000
9	battery	35	0.000000	0.000000	0.000000
10	value	34	0.000000	0.000000	0.000000
11	ear	34	0.000000	0.000000	0.000000

```
# function to analyze the reviews
def getAnalysis(score):
    if score < 0:
        return 'Negative'
    elif score == 0:
        return 'Neutral'
    else:
        return 'Positive'

wd_df['Analysis'] = wd_df['Polarity'].apply(getAnalysis)
```

wd_df

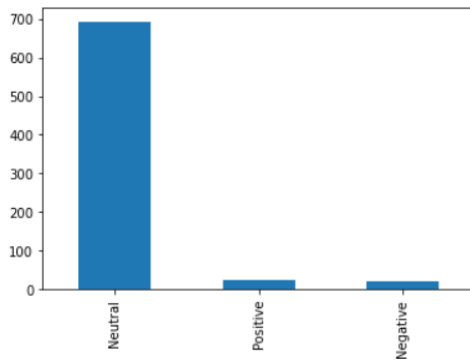
	words	count	sentiment	Subjectivity	Polarity	Analysis
0	product	326	0.000000	0.000000	0.000000	Neutral
1	quality	240	0.000000	0.000000	0.000000	Neutral
2	sound	128	0.400000	0.400000	0.400000	Positive
3	headphone	109	0.000000	0.000000	0.000000	Neutral
4	buy	61	0.000000	0.000000	0.000000	Neutral
5	price	60	0.000000	0.000000	0.000000	Neutral
6	money	56	0.000000	0.000000	0.000000	Neutral
7	bass	40	-0.150000	0.500000	-0.150000	Negative
8	month	38	0.000000	0.000000	0.000000	Neutral
9	battery	35	0.000000	0.000000	0.000000	Neutral
10	value	34	0.000000	0.000000	0.000000	Neutral
11	ear	34	0.000000	0.000000	0.000000	Neutral

```
wd_df['Analysis'].count
```

```
<bound method Series.count of 0      Neutral
1      Neutral
2      Positive
3      Neutral
4      Neutral
5      Neutral
6      Neutral
7      Negative
8      Neutral
9      Neutral
10     Neutral
11     Neutral
12     Neutral
13     Neutral
14     Neutral
15     Neutral
16     Neutral
17     Neutral
18     Neutral
```

```
wd_df['Analysis'].value_counts().plot(kind='bar')
```

<AxesSubplot:>



➤ Sentiment Analysis for each review

```
amazon['reviews'][:5].apply(lambda x: TextBlob(x).sentiment)
```

```
0          (0.7, 1.0)
1      (0.25125, 0.5925)
2      (0.07999999999999996, 0.8)
3          (-0.4, 0.4)
4      (0.21721428571428567, 0.5209404761904762)
Name: reviews, dtype: object
```

```
amazon['sentiment'] = amazon['reviews'].apply(lambda x: TextBlob(x).sentiment[0] )
amazon[['reviews', 'sentiment']]
```

	reviews	sentiment
0	i happy buy product... nice product.....	7.000000e-01
1	amazingly amazon it say 8 hr since i use watch...	2.512500e-01
2	base king base really good. value range money....	8.000000e-02
3	the head phone plastic band broken out. this p...	-4.000000e-01
4	cheap plastic, average product. sound distorte...	2.172143e-01
5	value money sound crystal clearbass pretty goo...	3.250000e-01
6	don't buy please... bass disgusting... range g...	-1.500000e-01
7	cheap quality i used 1 month headphone literal...	1.114583e-01
8	after 2 month review marvelous performanceif w...	4.277778e-01
9	broken within one month!!! i huge fan boat pur...	-1.802083e-01
10	product great recieved used product instead br...	1.602968e-01
11	never go boat product didn't last month.... it ...	-7.142857e-02

Subjectivity and Polarity

```
from textblob import TextBlob
def getSubjectivity(clean_review):
    return TextBlob(clean_review).sentiment.subjectivity

def getPolarity(clean_review):
    return TextBlob(clean_review).sentiment.polarity

amazon['Subjectivity'] = amazon['reviews'].apply(getSubjectivity)
amazon['Polarity'] = amazon['reviews'].apply(getPolarity)
```

amazon

	rating	reviews	sentiment	Subjectivity	Polarity
0	5.0	i happy buy product... nice product.....	7.000000e-01	1.000000	7.000000e-01
1	4.0	amazingly amazon it say 8 hr since i use watch...	2.512500e-01	0.592500	2.512500e-01
2	4.0	base king base really good. value range money....	8.000000e-02	0.800000	8.000000e-02
3	1.0	the head phone plastic band broken out. this p...	-4.000000e-01	0.400000	-4.000000e-01
4	2.0	cheap plastic, average product. sound distorte...	2.172143e-01	0.520940	2.172143e-01
5	5.0	value money sound crystal clearbass pretty goo...	3.250000e-01	0.700000	3.250000e-01
6	1.0	don't buy please... bass disgusting... range g...	-1.500000e-01	0.700000	-1.500000e-01
7	1.0	cheap quality i used 1 month headphone literal...	1.114583e-01	0.516667	1.114583e-01
8	4.0	after 2 month review marvelous performanceif w...	4.277778e-01	0.607407	4.277778e-01
9	1.0	broken within one month!!! i huge fan boat pur...	-1.802083e-01	0.625000	-1.802083e-01
10	4.0	product great recieved used product instead br...	1.602968e-01	0.484323	1.602968e-01
11	1.0	never go boat product didn't last month.... it ...	-7.142857e-02	0.219048	-7.142857e-02

```
# function to analyze the reviews
def getAnalysis(score):
    if score < 0:
        return 'Negative'
    elif score == 0:
        return 'Neutral'
    else:
        return 'Positive'

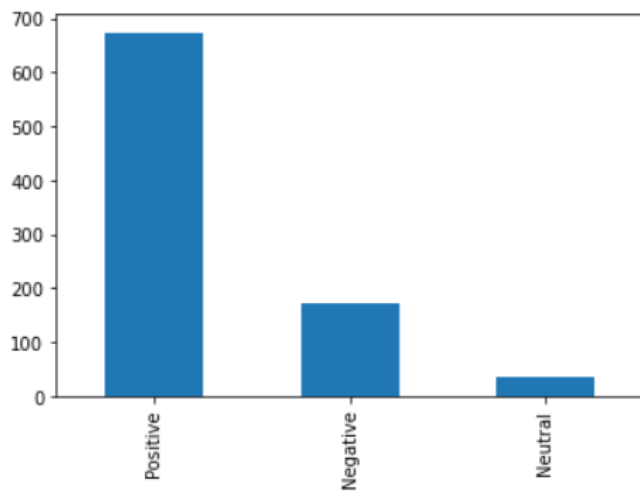
amazon['Analysis'] = amazon['Polarity'].apply(getAnalysis)
```

amazon

	rating	reviews	sentiment	Subjectivity	Polarity	Analysis
0	5.0	i happy buy product... nice product.....	7.000000e-01	1.000000	7.000000e-01	Positive
1	4.0	amazingly amazon it say 8 hr since i use watch...	2.512500e-01	0.592500	2.512500e-01	Positive
2	4.0	base king base really good. value range money....	8.000000e-02	0.800000	8.000000e-02	Positive
3	1.0	the head phone plastic band broken out. this p...	-4.000000e-01	0.400000	-4.000000e-01	Negative
4	2.0	cheap plastic, average product. sound distorte...	2.172143e-01	0.520940	2.172143e-01	Positive
5	5.0	value money sound crystal clearbass pretty goo...	3.250000e-01	0.700000	3.250000e-01	Positive
6	1.0	don't buy please... bass disgusting... range g...	-1.500000e-01	0.700000	-1.500000e-01	Negative
7	1.0	cheap quality i used 1 month headphone literal...	1.114583e-01	0.516667	1.114583e-01	Positive
8	4.0	after 2 month review marvelous performanceif w...	4.277778e-01	0.607407	4.277778e-01	Positive
9	1.0	broken within one month!!! i huge fan boat pur...	-1.802083e-01	0.625000	-1.802083e-01	Negative
10	4.0	product great recieved used product instead br...	1.602968e-01	0.484323	1.602968e-01	Positive
11	1.0	never go boat product didn't last month.... it ...	-7.142857e-02	0.219048	-7.142857e-02	Negative

```
amazon['Analysis'].value_counts().plot(kind='bar')
```

<AxesSubplot:>



amazon

	rating	reviews	sentiment	Subjectivity	Polarity	Analysis
0	5.0	i happy buy product... nice product....	7.000000e-01	1.000000	7.000000e-01	Positive
1	4.0	amazingly amazon it say 8 hr since i use watch...	2.512500e-01	0.592500	2.512500e-01	Positive
2	4.0	base king base really good. value range money....	8.000000e-02	0.800000	8.000000e-02	Positive
3	1.0	the head phone plastic band broken out. this p...	-4.000000e-01	0.400000	-4.000000e-01	Negative
4	2.0	cheap plastic, average product. sound distorte...	2.172143e-01	0.520940	2.172143e-01	Positive
5	5.0	value money sound crystal clearbass pretty goo...	3.250000e-01	0.700000	3.250000e-01	Positive
6	1.0	don't buy please... bass disgusting... range g...	-1.500000e-01	0.700000	-1.500000e-01	Negative
7	1.0	cheap quality i used 1 month headphone literal...	1.114583e-01	0.516667	1.114583e-01	Positive
8	4.0	after 2 month review marvelous performanceif w...	4.277778e-01	0.607407	4.277778e-01	Positive
9	1.0	broken within one month!!! i huge fan boat pur...	-1.802083e-01	0.625000	-1.802083e-01	Negative
10	4.0	product great recieved used product instead br...	1.602968e-01	0.484323	1.602968e-01	Positive
11	1.0	never go boat product didn't last month.... it ...	-7.142857e-02	0.219048	-7.142857e-02	Negative

Generate Positive Reviews Word Cloud

```
from wordcloud import WordCloud
wc = WordCloud(width=500,height=500,min_font_size=10,max_words=300,background_color='black')
```

```
Positive = wc.generate(amazon[amazon['Polarity']>0]['reviews'].str.cat(sep=""))
```

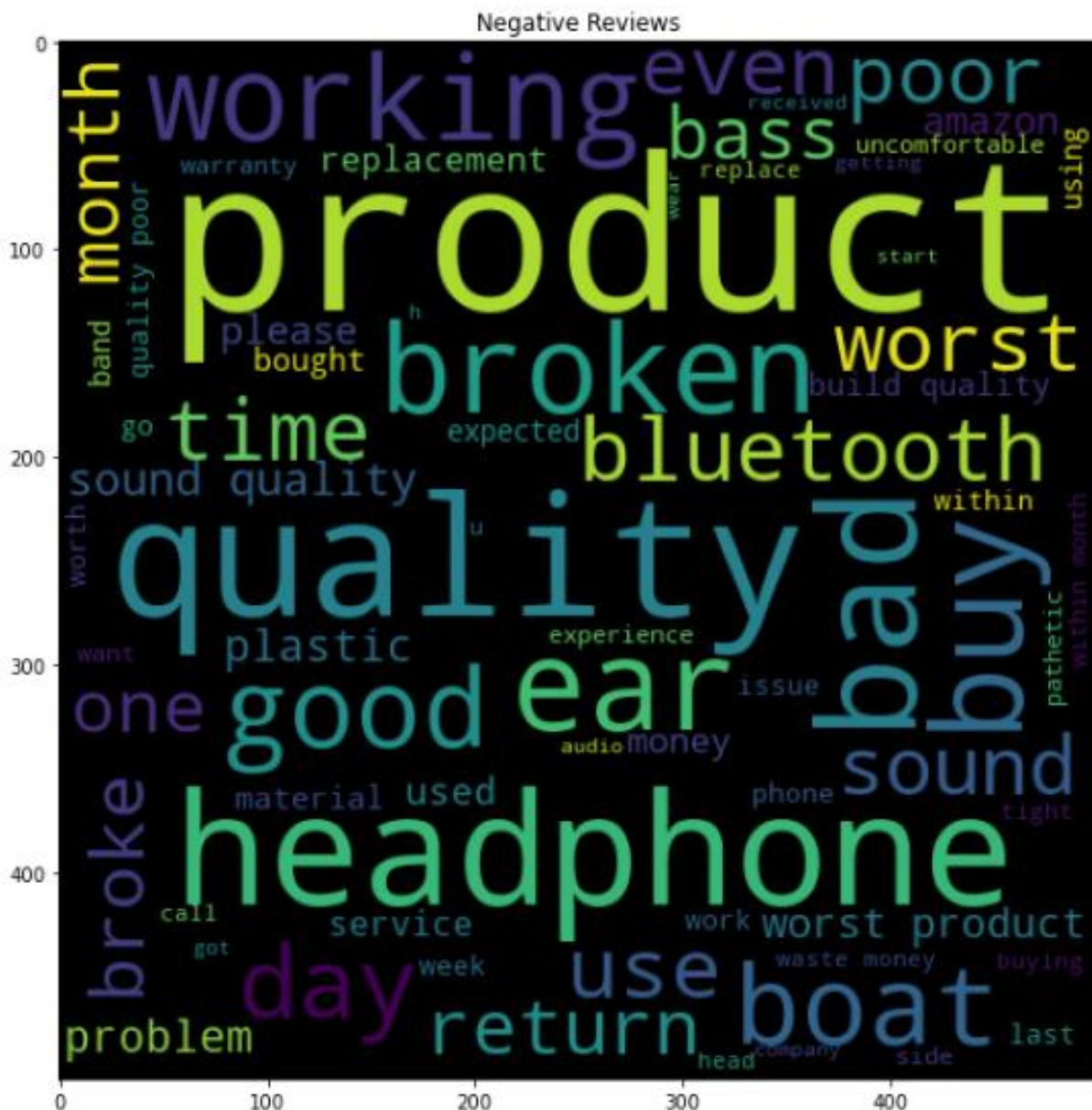
```
plt.figure(figsize=(10,10))
plt.imshow(Positive)
plt.title('Positive Reviews')
plt.show()
```



Generate Negative Reviews Word Cloud

```
] : Negative=wc.generate(amazon[amazon['Polarity']<0]['reviews'].str.cat(sep=""))
```

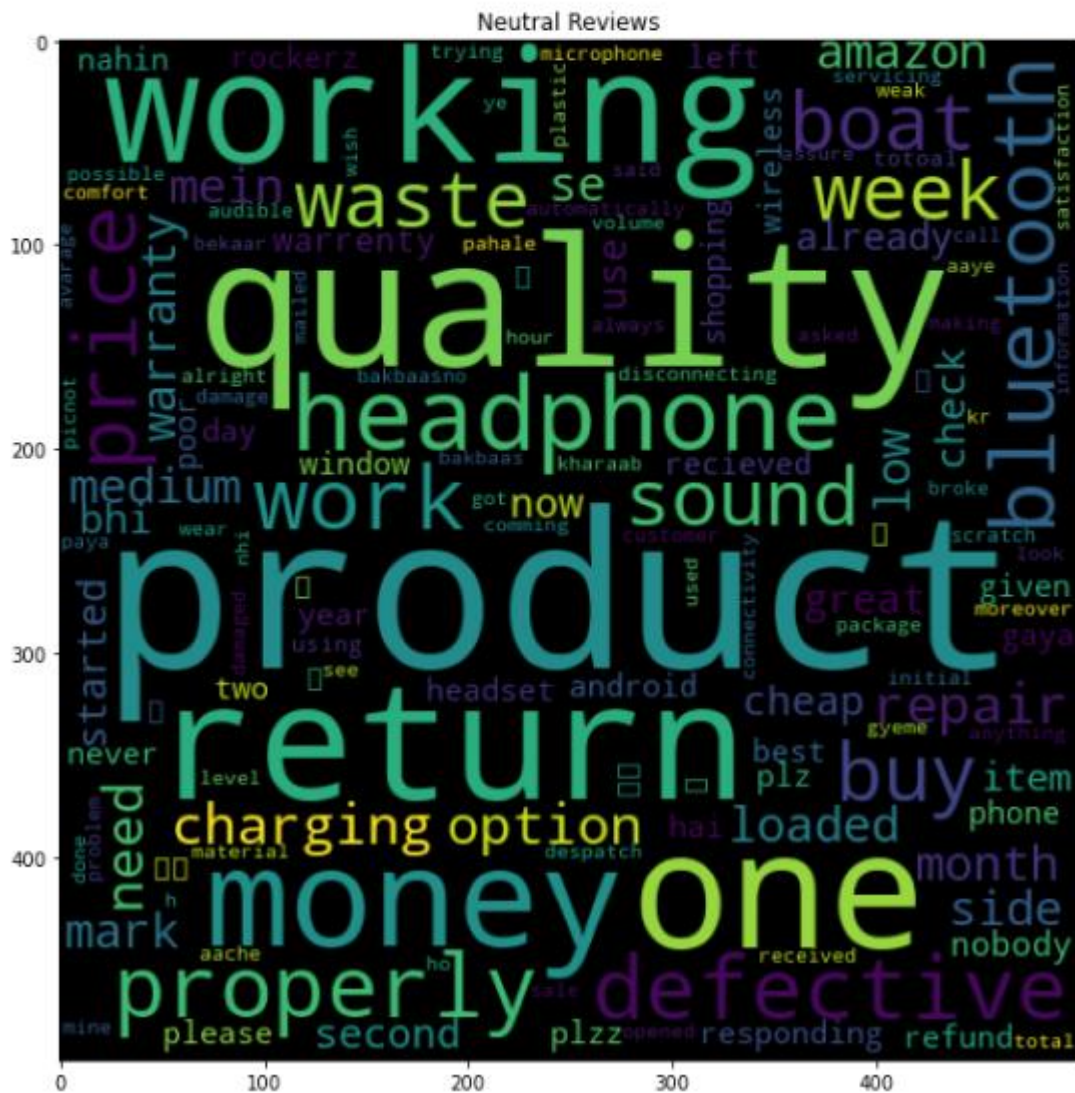
```
]: plt.figure(figsize=(10,10))
plt.imshow(Negative)
plt.title('Negative Reviews')
plt.show()
```



Generate Neutral Reviews Word Cloud

```
: Neutral = wc.generate(amazon[amazon['Polarity']==0]['reviews'].str.cat(sep=""))
```

```
: plt.figure(figsize=(10,10))
plt.imshow(Neutral)
plt.title('Neutral Reviews')
plt.show()
```



IMPLEMENTATION

➤ **MODEL BUILDING :**

➤ How to Build a Machine Learning Model

- ❖ Dataset. A dataset is the starting point in your journey of building the machine learning model. Simply put, the dataset...
- ❖ Exploratory Data Analysis. Exploratory data analysis (EDA) is performed in order to gain a preliminary understanding and...
- ❖ Data Pre-Processing. Data pre-processing (also known as data cleaning, data wrangling...

Sk-Learn :

Sklearn (scikit-learn) is a Python library that provides a wide range of unsupervised and supervised machine learning algorithms.

It is also one of the most used machine learning libraries and is built on top of SciPy.

What is Sklearn used for?

The Sklearn Library is mainly used for modeling data and it provides efficient tools that are easy to use for any kind of predictive data analysis.

The main use cases of this library can be categorized into 6 categories which are the following:

- Preprocessing
- Regression
- Classification
- Clustering
- Model Selection
- Dimensionality Reduction

As this article is mainly aimed at beginners, we will stick to the core concepts of each category and explore some of its most popular features and algorithms.

Advanced readers can use this article as a recollection of some of the main use cases and intuitions behind popular sklearn features that most ML practitioners couldn't live without.

Each category will be explained in a beginner-friendly and illustrative way followed by the most used models, the intuition behind them, and hands-on experience. But first, we need to set up our sklearn library.

Shape :

Measures of shape describe the distribution (or pattern) of the data within a dataset. The distribution shape of quantitative data can be described as there is a logical order to the values, and the 'low' and 'high' end values on the x-axis of the histogram are able to be identified.

```
: from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
cv=CountVectorizer()
vectorizer=TfidfVectorizer(max_features=10000)

: x=vectorizer.fit_transform(amazon['reviews'])

: #rows, reviews
x.shape

: (880, 2849)
```

LabelEncoder for classification Model :

Use LabelEncoder for label columns in case of supervised learning when it is binary classification problem. Don't use LabelEncoder when the categorical features have more than two values. The nominal categorical features having more than two values may get treated as ordinal one by the machine learning model. Although model performance won't suffer much, it is recommended to use One-Hot encoder for categorical features having more than two different types of value.

LabelEncoder for classification Model

```
: from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()

: amazon['target']=encoder.fit_transform(amazon['Analysis'])
amazon
```

	rating	reviews	sentiment	Subjectivity	Polarity	Analysis	target
0	5.0	i happy buy product... nice product....	7.000000e-01	1.000000	7.000000e-01	Positive	2
1	4.0	amazingly amazon it say 8 hr since i use watch...	2.512500e-01	0.592500	2.512500e-01	Positive	2
2	4.0	base king base really good. value range money....	8.000000e-02	0.800000	8.000000e-02	Positive	2
3	1.0	the head phone plastic band broken out. this p...	-4.000000e-01	0.400000	-4.000000e-01	Negative	0
4	2.0	cheap plastic, average product. sound distorte...	2.172143e-01	0.520940	2.172143e-01	Positive	2
5	5.0	value money sound crystal clearbass pretty goo...	3.250000e-01	0.700000	3.250000e-01	Positive	2
6	1.0	don't buy please... bass disgusting... range g...	-1.500000e-01	0.700000	-1.500000e-01	Negative	0
7	1.0	cheap quality i used 1 month headphone literal...	1.114583e-01	0.516667	1.114583e-01	Positive	2
8	4.0	after 2 month review marvelous performanceif w...	4.277778e-01	0.607407	4.277778e-01	Positive	2
9	1.0	broken within one month!!! i huge fan boat pur...	-1.802083e-01	0.625000	-1.802083e-01	Negative	0
10	4.0	product great recieved used product instead br...	1.602968e-01	0.484323	1.602968e-01	Positive	2
11	1.0	never go boat product didn't last month... it ...	-7.142857e-02	0.219048	-7.142857e-02	Negative	0

KNN :

- ☐ The k-nearest neighbors (KNN) algorithm is a simple, supervised machine learning algorithm that can be used to solve both classification and regression problems.
- ☐ K-nearest neighbours (KNN) algorithm is a type of supervised ML algorithm which can be used for both classification as well as regression predictive problems. However, it is mainly used for classification predictive problems in industry.
- ☐ K Nearest Neighbor is one of the fundamental algorithms in machine learning. Machine learning models use a set of input values to predict output values. KNN is one of the simplest forms of machine learning algorithms mostly used for classification. It classifies the data point on how its neighbor is classified.
- ☐ KNN classifies the new data points based on the similarity measure of the earlier stored data points. For example, if we have a dataset of tomatoes and bananas. KNN will store similar measures like shape and color. When a new object comes it will check its similarity with the color (red or yellow) and shape.

When is KNN?

- ☐ We have properly labeled data. For example, if we are predicting someone is having diabetes or not the final label can be 1 or 0. It cannot be NaN or -1.
- ☐ Data is noise-free. For the diabetes data set we cannot have a Glucose level as 0 or 10000. It's practically impossible.

➤ Small dataset.

- ☐ We usually use Euclidean distance to calculate the nearest neighbor. If we have two points (x, y) and (a, b) . The formula for Euclidean distance (d) will be $d = \sqrt{(x-a)^2 + (y-b)^2}$
- ☐ We can see that a user listens in average to 41.5 songs. Even the maximum amount of songs listened by a user is 395, and we have 1079 songs in our dataset.

- So, not all user listen to all songs, so a lot of values in the song x users matrix are going to be zero. Thus, we'll be dealing with extremely sparse data.
- How sparse? Let's check that:
- Get how many values should it be if all songs have been listen by all users. #
Subtract the total values with the actual shape of the DataFrame songs

KNN

```
from sklearn.model_selection import KFold
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier()
from sklearn.model_selection import GridSearchCV
k_range = list(range(1, 31))
param_grid = dict(n_neighbors=k_range)

# defining parameter range
grid = GridSearchCV(knn, param_grid, cv=10, scoring='accuracy', return_train_score=False, verbose=1)

# fitting the model for grid search
grid_search=grid.fit(X_train, y_train)
```

Fitting 10 folds for each of 30 candidates, totalling 300 fits

```
print(grid_search.best_params_)

{'n_neighbors': 12}
```

```
accuracy = grid_search.best_score_ *100
print("Accuracy for our training dataset with tuning is : {:.2f}%".format(accuracy) )
```

Accuracy for our training dataset with tuning is : 82.06%

```
#Create KNN Classifier
knn = KNeighborsClassifier(n_neighbors=7)
```

```
#Train the model using the training sets
knn.fit(X_train, y_train)
```

KNeighborsClassifier(n_neighbors=7)

```
#Predict the response for test dataset
y_pred = knn.predict(X_test)
```

```
#Import scikit-learn metrics module for accuracy calculation
from sklearn import metrics
# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred)*100)
```

Accuracy: 90.9090909090909

➤ **Bagging & Boosting :**

- ❖ **Bagging:** It is a homogeneous weak learners' model that learns from each other independently in parallel and combines them for determining the model average.
- ❖ Bootstrap Aggregating, also known as bagging, is a machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in statistical classification and regression. It decreases the variance and helps to avoid overfitting. It is usually applied to decision tree methods. Bagging is a special case of the model averaging approach.

➤ **Description of the Technique**

- ❖ Suppose a set D of d tuples, at each iteration i , a training set D_i of d tuples is selected via row sampling with a replacement method (i.e., there can be repetitive elements from different d tuples) from D (i.e., bootstrap). Then a classifier model M_i is learned for each training set $D < i$. Each classifier M_i returns its class prediction. The bagged classifier M^* counts the votes and assigns the class with the most votes to X (unknown sample).

➤ **Implementation Steps of Bagging**

- **Step 1:** Multiple subsets are created from the original data set with equal tuples, selecting observations with replacement.
- **Step 2:** A base model is created on each of these subsets.
- **Step 3:** Each model is learned in parallel with each training set and independent of each other.
- **Step 4:** The final predictions are determined by combining the predictions from all the models.

➤ **Example of Bagging**

- ❖ The Random Forest model uses Bagging, where decision tree models with higher variance are present. It makes random feature selection to grow trees. Several random trees make a Random Forest.

➤ **Boosting:**

- ❖ **Boosting** - It is also a homogeneous weak learners' model but works differently from Bagging. In this model, learners learn sequentially and adaptively to improve model predictions of a learning algorithm.

- ❖ Boosting is an ensemble modeling technique that attempts to build a strong classifier from the number of weak classifiers. It is done by building a model by using weak models in series. Firstly, a model is built from the training data. Then the second model is built which tries to correct the errors present in the first model. This procedure is continued and models are added until either the complete training data set is predicted correctly or the maximum number of models is added.

➤ Boosting Algorithms

- ❖ There are several boosting algorithms. The original ones, proposed by Robert Schapire and Yoav Freund were not adaptive and could not take full advantage of the weak learners. Schapire and Freund then developed AdaBoost, an adaptive boosting algorithm that won the prestigious Gödel Prize. AdaBoost was the first really successful boosting algorithm developed for the purpose of binary classification. AdaBoost is short for Adaptive Boosting and is a very popular boosting technique that combines multiple “weak classifiers” into a single “strong classifier”.

Algorithm:

1. Initialise the dataset and assign equal weight to each of the data point.
2. Provide this as input to the model and identify the wrongly classified data points.
3. Increase the weight of the wrongly classified data points and decrease the weights of correctly classified data points. And then normalize the weights of all data points.
4. if (got required results)
 Goto step 5
 else
 Goto step 2
5. End

➤ Similarities Between Bagging and Boosting

- ❖ Bagging and Boosting, both being the commonly used methods, have a universal similarity of being classified as ensemble methods. Here we will explain the similarities between them.
 1. Both are ensemble methods to get N learners from 1 learner.
 2. Both generate several training data sets by random sampling.
 3. Both make the final decision by averaging the N learners (or taking the majority of them i.e Majority Voting).
 4. Both are good at reducing variance and provide higher stability.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score#importing metrics for accuracy calculation (confusion matrix)
from sklearn.ensemble import BaggingClassifier#bagging combines the results of multipls models to get a generalized result.
from sklearn.ensemble import AdaBoostClassifier #boosting method attempts to correct the errors of previous models.
from sklearn.metrics import classification_report, confusion_matrix
```

```
dcmodel = BaggingClassifier(DecisionTreeClassifier(max_depth = 6), random_state=0) #decision tree classifier object
dcmodel = AdaBoostClassifier(DecisionTreeClassifier(max_depth = 6), random_state=0) #decision tree classifier object
```

```
dcmodel = dcmodel.fit(X_train,y_train) #train decision tree
y_predict = dcmodel.predict(X_test)
```

```
print("Accuracy : ", accuracy_score(y_test,y_predict)*100 )
```

```
Accuracy : 86.36363636363636
```

➤ Random Forest :

- ❖ Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of ensemble learning, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.
- ❖ As the name suggests, "Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- ❖ The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
```

```
rf = RandomForestClassifier(n_jobs=3,oob_score=True,n_estimators=15,criterion="entropy")
```

```
rf.fit(X_train,y_train) # Fitting RandomForestClassifier model from sklearn.ensemble
rf.estimators_ #
rf.classes_ # class labels (output)
rf.n_classes_ # Number of levels in class labels
rf.n_features_ # Number of input features in model 8 here.

rf.n_outputs_ # Number of outputs when fit performed

rf.oob_score_
```

```
0.7916666666666666
```

```
rf.predict(X_test)
```

```
array([2, 0, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 2, 1,
        2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 0, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 0, 2, 0, 2, 2])
```

```
preds = rf.predict(X_test)
pd.Series(preds).value_counts()
```

```
2    76
0     9
1     3
dtype: int64
```

```
preds
```

```
array([2, 0, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 2, 2, 2, 2, 1,
        2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 0, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 0, 2, 0, 2, 2])
```


➤ CONFUSION METRICS :

- ❖ A confusion matrix is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known. The confusion matrix itself is relatively simple to understand, but the related terminology can be confusing.

```
: confusion_matrix = confusion_matrix(y_train,y_pred)
confusion_matrix
```

```
: array([[117,   0,  41],
        [  4,   0,  28],
        [  0,   0, 602]], dtype=int64)
```

```
: from sklearn.metrics import classification_report
print(classification_report(y_train,y_pred))
```

	precision	recall	f1-score	support
0	0.97	0.74	0.84	158
1	0.00	0.00	0.00	32
2	0.90	1.00	0.95	602
accuracy			0.91	792
macro avg	0.62	0.58	0.59	792
weighted avg	0.87	0.91	0.89	792

➤ **Logistic Regression giving the best result :**

- ❖ Logistic regression becomes a classification technique only when a decision threshold is brought into the picture. The setting of the threshold value is a very important aspect of Logistic regression and is dependent on the classification problem itself.
- ❖ The decision for the value of the threshold value is majorly affected by the values of precision and recall. Ideally, we want both precision and recall to be 1, but this seldom is the case.

➤ **NLP :**

- ❖ Natural language processing (NLP) is a subfield of Artificial Intelligence (AI). This is a widely used technology for personal assistants that are used in various business fields/areas. This technology works on the speech provided by the user, breaks it down for proper understanding and processes accordingly. This is a very recent and effective approach due to which it has a really high demand in today's market. Natural Language Processing is an upcoming field where already many transitions such as compatibility with smart devices, interactive talks with a human have been made possible. Knowledge representation, logical reasoning, and constraint satisfaction were the emphasis of AI applications in NLP. Here first it was applied to semantics and later to the grammar. In the last decade, a significant change in NLP research has resulted in the widespread use of statistical approaches such as machine learning and data mining on a massive scale. The need for automation is never ending courtesy of the amount of work required to be done these days. NLP is a very favourable, but aspect when it comes to automated applications. The applications of NLP have led it to be one of the most sought-after methods of implementing machine learning. Natural Language Processing (NLP) is a field that combines computer science, linguistics, and machine learning to study how computers and humans communicate in natural language. The goal of NLP is for computers to be able to interpret and generate human language. This not only improves the efficiency of work done by humans but also helps in interacting with the machine. NLP bridges the gap of interaction between humans and electronic devices.

DIPLOYMANT

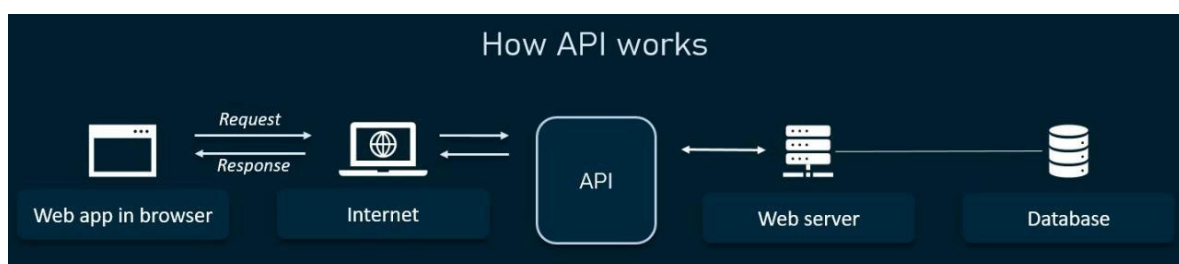
API:

Application programming interfaces, or APIs, simplify software development and innovation by enabling applications to exchange data and functionality easily and securely.

An API is a set of defined rules that explain how computers or applications communicate with one another. APIs sit between an application and the web server, acting as an intermediary layer that processes data transfer between systems.

Here's how an API works:

- A client application initiates an API call to retrieve information—also known as a request. This request is processed from an application to the web server via the API's Uniform Resource Identifier (URI) and includes a request verb, headers, and sometimes, a request body.
- After receiving a valid request, the API makes a call to the external program or web server.
- The server sends a response to the API with the requested information.
- The API transfers the data to the initial requesting application.



```

Users > HP > Downloads > Amazone College > dump file > app.py > ...
1  # -*- coding: utf-8 -*-
2  """
3  Created on Sat Apr 24 21:49:27 2022
4
5  @author: 91960
6  """
7
8  from bs4 import BeautifulSoup
9  # import urllib.request as url
10 import requests
11 import streamlit as st
12 import statistics as sts
13 import pickle
14 import pandas as pd
15 import seaborn as sns
16 import matplotlib.pyplot as plt
17
18
19
20
21 # so we can see the output side by side
22 st.set_page_config(layout="wide")
23
24 #load model
25 def load_model():
26     with open('C:/Users/HP/Downloads/Amazone College/dump file/model.pkl', 'rb') as file:
27         model = pickle.load(file)
28     return model
29 model = load_model()
30
31 #load tf_idf vectorizer
32 vectorizer = pickle.load(open('C:/Users/HP/Downloads/Amazone College/dump file/vectorizer.pkl', "rb"))
33
34

```

```

def get_soup(url):
    #url = url
    page = requests.get(url)
    soup = BeautifulSoup(page.content, 'html.parser')
    return soup

reviewlist = []
#fetching only data which is necessary like rating title and content given by the customers
def get_reviews(soup):
    reviews = soup.find_all('div', {'data-hook': 'review'})
    try:
        for item in reviews:
            review = {
                'title': item.find('a', {'data-hook': 'review-title'}).text.strip(),
                'rating': float(item.find('i', {'data-hook': 'review-star-rating'}).text.replace('out of 5 stars', '').strip()),
                'content': item.find('span', {'data-hook': 'review-body'}).text.strip(),
            }
            reviewlist.append(review)
    except:
        pass

```

```

# compare easier
st.title("AMAZON PRODUCT REVIEWS")
# col1,col2 = st.columns(2)
# with col1:
#     url = input('URL link to scrape')
urls = st.text_input("enter the page url , you want to get the data")
#creating a loop from 1 to 100 reviews pages of the product
for x in range(1,100):
    soup = get_soup(urls)
    get_reviews(soup)
    if not soup.find('li', {'class': 'a-disabled a-last'}):
        pass
    else:
        break

df = pd.DataFrame(reviewlist)
df['reviews'] = df['title']+" "+df['content']
df['reviews']

col1,col2 = st.columns(2)
with col1:
    st.subheader("PRODUCT SENTIMENT")
    ok = st.button("get sentiment")
if ok:
    test = vectorizer.transform(df['reviews'])
    prediction = model.predict(test)
    res = sts.mode(prediction)
    if res == 2:
        st.write("positive sentiment")
        #st.image(posit,caption='positive sentiment')
    elif res == 1:
        st.write("nutral sentiment")
    else :
        st.write("negative sentiment")

```

```

with col2:
    #for show the accurance of sentiments
    d = {'results':prediction}
    pred = pd.DataFrame(data=d)
    def lable(score):
        if score == 0:
            return 'Negative'
        elif score == 1:
            return 'Neutral'
        else:
            return 'Positive'
    pred['lable'] = pred['results'].apply(lable)
    def countplot():
        fig = plt.figure(figsize=(5,5))
        sns.countplot(pred['lable'])
        st.pyplot(fig)
    st.subheader("COUNT THE SENTIMENTS")
    ok = st.button("Reviews countplot")
    if ok:
        countplot()

```

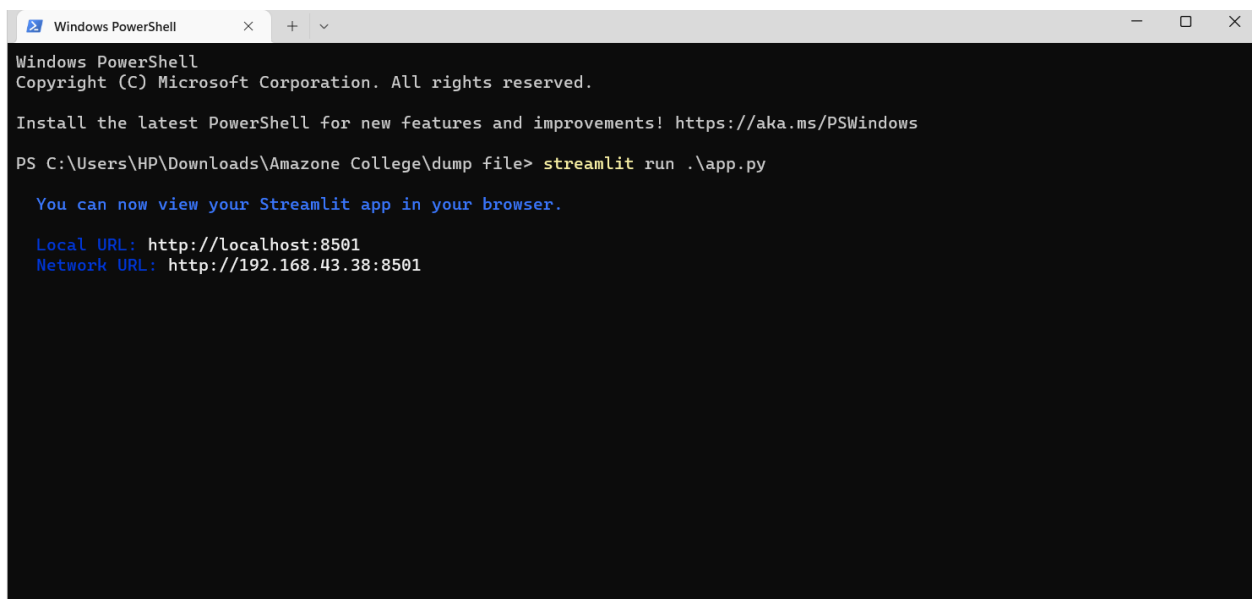
Streamlit :

- Streamlit is an open-source python library for creating and sharing web apps for data science and machine learning projects. The library can help you create and deploy your data science solution in a few minutes with a few lines of code.
- Streamlit can seamlessly integrate with other popular python libraries used in Data science such as NumPy, Pandas, Matplotlib, Scikit-learn and many more.
- Note: Streamlit uses React as a frontend framework to render the data on the screen.

It Provide two type of network to host :

Local URL

Network URL

A screenshot of a Windows PowerShell terminal window. The title bar shows 'Windows PowerShell' with standard window controls. The terminal text includes: 'Windows PowerShell', 'Copyright (C) Microsoft Corporation. All rights reserved.', 'Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows', the command 'PS C:\Users\HP\Downloads\Amazone College\dump file> streamlit run .\app.py', and the output 'You can now view your Streamlit app in your browser.' followed by 'Local URL: http://localhost:8501' and 'Network URL: http://192.168.43.38:8501'.

- It will host direct to the server.
- User search any type of product from E-commerce websites link in search bar.

HOME :

AMAZON PRODUCT REVIEWS

enter the page url , you want to get the data

Output :

AMAZON PRODUCT REVIEWS

enter the page url , you want to get the data

https://www.amazon.in/Rockerz-370-Headphone-Bluetooth-Lightweight/dp/B0856HNMR7/ref=sr_1_1_ss

	reviews
0	Mic problem I must say everything thing is good and all rounder EXCEPT MIC in this i will give 0 🌟 becoz I have
1	Worth to buy in this price range Sound quality is good compare to price. Light weight product. Super battery ba
2	Sound quality Sound quality is good... Compare to price noice and md bass is good Read more
3	True things about this headphone:- Everything is the best In this headphones, but yes the build quality could in
4	Great for online meetings Great for online meetings and good for little binge watching with in rupees 750 i have
5	Good Quality in lowest price Good quality in this price with compare to other one. But It's quality can't be comp
6	Average product not comfortable This headphone has good battery backup, good range, good sound but not fo
7	Good at this price..... Looks good, bass is just ok, not much comfort but sound is good. Microphone for calling is
8	Mic problem I must say everything thing is good and all rounder EXCEPT MIC in this i will give 0 🌟 becoz I have
9	Worth to buy in this price range Sound quality is good compare to price. Light weight product. Super battery ba

It give positive sentiment :

PRODUCT SENTIMENT

get sentiment

positive sentiment

COUNT THE SENTIMENTS

Reviews countplot

Another product link – [Redmi 9 Activ \(Coral Green, 4GB RAM, 64GB Storage\)| Octa-core Helio G35 | 5000mAh Battery : Amazon.in: Electronics](https://www.amazon.in/Redmi-Activ-Coral-Green-Storage/dp/B09GFLFMPS/ref=sr_1_1?crid=HXL8VRDFWN0T&key)

Output :

AMAZON PRODUCT REVIEWS

enter the page url , you want to get the data

https://www.amazon.in/Redmi-Activ-Coral-Green-Storage/dp/B09GFLFMPS/ref=sr_1_1?crid=HXL8VRDFWN0T&key

	reviews
0	Worth it money Camera is ok ,Battery is good ,Speed is ok ,Look is also ok,But only one thing is bad and that is :
1	Overall good Nice performance but it's display in not come with any type of protection 🧊 I first time dropped n
2	Redmi 9 activ A good mobile in this price range,for people using WhatsApp, Facebook, Instagram and YouTube,t
3	The last camera is not there You might not believe me but if you zoom from another device you will be able to s
4	Phone is good to use but is little slow.... Can be used as secondary phone..... Okay to be used if be utilized as se
5	Nice one At this price go for it.. good phone... Better to go for other companies than samsung, disgusting phone
6	Worth to Buy under Rs 8000 After buying the mobile uninstall all unnecessary system apps using ADB commanc
7	Everything ok but Everything ok but..Heating is the problem.. Redmi should solve this heating problem. Read n
8	Worth it money Camera is ok ,Battery is good ,Speed is ok ,Look is also ok,But only one thing is bad and that is :
9	Overall good Nice performance but it's display in not come with any type of protection 🧊 I first time dropped n

PRODUCT SENTIMENT

get sentiment

positive sentiment

COUNT THE SENTIMENTS

Reviews countplot

CONCLUSION

- As the results on the test data shows, LSTM networks are the most suitable for binary sentiment analysis on Amazon.com product reviews. Based on the results on the evaluation datasets, we can conclude that LSTM performs very well (accuracy > 0.90) for binary classification, and that does not depend strongly on the type of product where the reviews come from. As it can be seen clearly from confusion matrices in Figure 2, the LSTM network both performs accurate results for positive and negative classes. Since the training dataset is also balanced, getting balanced results from both classes shows the model's reliability. In conclusion, LSTM networks are very suitable for classification of the sentiment on product reviews. These results do not depend on the type of product where the reviews are given to. Since we get ~ 90% accuracy with just training 1% of data, we can say that sentimental analysis methods are feasible with Amazon review data.
- Online shopping is a source of vast unstructured and noisy data sets that can be processed to locate interesting patterns and trends. Apache Spark proved prolific in extracting live streams of data and has further capability to store batches of data in HDFS and other major conventional storages. The processing capabilities of Spark makes the project flexible to further extend to multiple nodes, thereby supporting distributed computing. Real time data analysis makes it possible for business organizations to keep track of their services and generates opportunities to promote, advertise and improve from time to time.

FUTURE SCOPE

- ❖ We leave refinement of the narratives for accuracy as a part for future work and are also working to incorporate principal component analysis (PCA) that will automate the active learning process of our system and will fully automate the data labeling process and will require less assistance. We will be looking into datasets obtained from local supermarket. This will provide better understanding of our sentiments based on demographics. And lastly we will try to work on this product to achieve a generalize form of this model.



Website can be hosted live.



Using ML & NLP concept we can Recommend very best product to user based on the sentimental analysis.



After the website is deployed ,it can be used by anyone and therefore the solution is feasible.

GitHub Links

- **Lakhan Patidar :** [lp401217 \(Lucky Patidar\) \(github.com\)](https://github.com/lp401217)
- **Sanjay kumar :**